

Guillermo Román

guillermo.roman@upm.es

Lars-Åke Fredlund

lfredlund@fi.upm.es

Manuel Carro

mcarro@fi.upm.es

Julio García

juliomanuel.garcia@upm.es

Tonghong Li

tonghong@fi.upm.es

- Fechas de entrega y penalización:

Hasta el martes 29 de octubre, 16:00 horas	0 %
Hasta el miércoles 30 de octubre, 16:00 horas	20 %
Hasta el jueves 31 de octubre, 16:00 horas	40 %

Después la puntuación máxima será 0
- Se comprobará plagio y se actuará sobre los detectados.
- Usad las horas de tutoría para preguntar sobre programación – son oportunidades excelentes para aprender.

Entrega

- Todos los ejercicios de laboratorio se deben entregar a través de
`http://deliverit.fi.upm.es`
- El/los fichero(s) que hay que subir es/son `Utils.java`,
`StackMachine.java`.
- La clase debe estar en el paquete `aed.recursion` .
- La documentación de la API de `aedlib.jar` está disponible en
`http://costa.ls.fi.upm.es/teaching/aed/docs/aedlib/`

Normas

- El tema principal de este laboratorio es practicar el uso de recursión.
- Por ese motivo está:
 - ▶ **Prohibido** usar bucles `for`, `for-each`, `do-while` o `while`.
 - ▶ **Prohibido** definir atributos adicionales en las clases a entregar.

Tareas

- El laboratorio tiene una parte obligatoria:
 - ▶ Calcular la raíz cuadrada $\sqrt{n}$ de un entero n (3 puntos).
 - ▶ Implementar la “Criba de Eratóstenes” (3 puntos).
 - ▶ Comprobar si dos listas de pares son iguales (4 puntos).
- Y un reto no obligatorio:
 - ▶ Implementar una “stack machine” (máquina de pila), inspirada en la ejecución de Java bytecode (3 puntos).
- En todos los casos usando recursión en lugar de bucles.

Tarea obligatoria

Calcular $\sqrt{n}$ de un entero $n > 0$ usando recursión

- Terminar la implementación del método `int sqrt(int n)` dentro la clase `Utils`.
- El método debe devolver la raíz cuadrada entera r de un número entero $n > 0$. El número r es el entero que cumple:
 - ▶ $r * r \leq n$
 - ▶ $(r + 1) * (r + 1) > n$
- Por ejemplo, si $n = 4$ entonces $r = 2$, y si $n = 5$ entonces $r = 2$.
- Un algoritmo simple puede probar enteros consecutivamente empezando en 1 hasta llegar a un i tal que $i * i > n$. Entonces $r = i - 1$.
- Para una puntuación más alta se puede usar una búsqueda binaria.

Tarea obligatoria

Implementar la “Criba de Eratóstenes”

Terminar la implementación del método

```
Iterable<Integer> primes(int n)
```

que debe generar y retornar una estructura iterable que devuelva todos los números primos en el intervalo $[2, n]$ en orden ascendente.

Un simulacro: buscar los números primos ≤ 69

Supondremos que tenemos un **boolean** asociado a cada uno de los números de la tabla a siguiente:

0	1	2	3	4	5	6	7	8	9
10	11	12	13	14	15	16	17	18	19
20	21	22	23	24	25	26	27	28	29
30	31	32	33	34	35	36	37	38	39
40	41	42	43	44	45	46	47	48	49
50	51	52	53	54	55	56	57	58	59
60	61	62	63	64	65	66	67	68	69

- Inicialmente todos los elementos $a[i] = \text{true}$

Un simulacro: buscar los números primos ≤ 69

0	1	2	3	4	5	6	7	8	9
10	11	12	13	14	15	16	17	18	19
20	21	22	23	24	25	26	27	28	29
30	31	32	33	34	35	36	37	38	39
40	41	42	43	44	45	46	47	48	49
50	51	52	53	54	55	56	57	58	59
60	61	62	63	64	65	66	67	68	69

- Empezamos en el elemento 2.
- Como $a[2] = \text{true}$ (es un número primo) asignamos a sus múltiplos $a[2 * 2], a[2 * 3], a[2 * 4], \dots$ el valor *false*.

Un simulacro: buscar los números primos ≤ 69

0	1	2	3	4	5	6	7	8	9
10	11	12	13	14	15	16	17	18	19
20	21	22	23	24	25	26	27	28	29
30	31	32	33	34	35	36	37	38	39
40	41	42	43	44	45	46	47	48	49
50	51	52	53	54	55	56	57	58	59
60	61	62	63	64	65	66	67	68	69

- Empezamos en el elemento 2.
- Como $a[2] = \text{true}$ (es un número primo) asignamos a sus múltiplos $a[2 * 2], a[2 * 3], a[2 * 4], \dots$ el valor *false*.

Un simulacro: buscar los números primos ≤ 69

0	1	2	3	4	5	6	7	8	9
10	11	12	13	14	15	16	17	18	19
20	21	22	23	24	25	26	27	28	29
30	31	32	33	34	35	36	37	38	39
40	41	42	43	44	45	46	47	48	49
50	51	52	53	54	55	56	57	58	59
60	61	62	63	64	65	66	67	68	69

- Continuamos con el 3.
- Como $a[3] = \text{true}$ (es un número primo), asignamos a sus múltiplos $a[3 * 2]$, $a[3 * 3]$, $a[3 * 4]$, ... el valor *false*.

Un simulacro: buscar los números primos ≤ 69

0	1	2	3	4	5	6	7	8	9
10	11	12	13	14	15	16	17	18	19
20	21	22	23	24	25	26	27	28	29
30	31	32	33	34	35	36	37	38	39
40	41	42	43	44	45	46	47	48	49
50	51	52	53	54	55	56	57	58	59
60	61	62	63	64	65	66	67	68	69

- Continuamos con el 3.
- Como $a[3] = \text{true}$ (es un número primo), asignamos a sus múltiplos $a[3 * 2]$, $a[3 * 3]$, $a[3 * 4]$, ... el valor *false*.

Un simulacro: buscar los números primos ≤ 69

0	1	2	3	4	5	6	7	8	9
10	11	12	13	14	15	16	17	18	19
20	21	22	23	24	25	26	27	28	29
30	31	32	33	34	35	36	37	38	39
40	41	42	43	44	45	46	47	48	49
50	51	52	53	54	55	56	57	58	59
60	61	62	63	64	65	66	67	68	69

- Continuamos con el 4.
- Como $a[4] = \text{false}$ (**no es un número primo**), lo ignoramos — sus múltiplos ya están “marcados”.

Un simulacro: buscar los números primos ≤ 69

0	1	2	3	4	5	6	7	8	9
10	11	12	13	14	15	16	17	18	19
20	21	22	23	24	25	26	27	28	29
30	31	32	33	34	35	36	37	38	39
40	41	42	43	44	45	46	47	48	49
50	51	52	53	54	55	56	57	58	59
60	61	62	63	64	65	66	67	68	69

- Continuamos con el 5.
- Como $a[5] = \text{true}$ (es un número primo), asignamos a sus múltiplos $a[5 * 2]$, $a[5 * 3]$, $a[5 * 4]$, ... el valor *false*.

Un simulacro: buscar los números primos ≤ 69

0	1	2	3	4	5	6	7	8	9
10	11	12	13	14	15	16	17	18	19
20	21	22	23	24	25	26	27	28	29
30	31	32	33	34	35	36	37	38	39
40	41	42	43	44	45	46	47	48	49
50	51	52	53	54	55	56	57	58	59
60	61	62	63	64	65	66	67	68	69

- Continuamos con el 5.
- Como $a[5] = \text{true}$ (es un número primo), asignamos a sus múltiplos $a[5 * 2]$, $a[5 * 3]$, $a[5 * 4]$, ... el valor *false*.

Un simulacro: buscar los números primos ≤ 69

0	1	2	3	4	5	6	7	8	9
10	11	12	13	14	15	16	17	18	19
20	21	22	23	24	25	26	27	28	29
30	31	32	33	34	35	36	37	38	39
40	41	42	43	44	45	46	47	48	49
50	51	52	53	54	55	56	57	58	59
60	61	62	63	64	65	66	67	68	69

- Continuamos de manera similar con 6 (no es un número primo), con 7 (que si lo es) y con 8 (no lo es).

Un simulacro: buscar los números primos ≤ 69

0	1	2	3	4	5	6	7	8	9
10	11	12	13	14	15	16	17	18	19
20	21	22	23	24	25	26	27	28	29
30	31	32	33	34	35	36	37	38	39
40	41	42	43	44	45	46	47	48	49
50	51	52	53	54	55	56	57	58	59
60	61	62	63	64	65	66	67	68	69

- El siguiente sería 9, pero $9 * 9 > 69$ y todos los factores de los números ≤ 69 ya han sido explorados: hemos terminado.
- Los números primos son los índices $i \geq 2$ para los que $a[i] = \text{true}$: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67.
- Creamos una estructura de datos y añadimos los números primos a la estructura.

El algoritmo de la “Criba de Eratóstenes” en pseudo-código

$p \leftarrow \text{boolean}[0 \dots n]$

▷ (an array of $n + 1$ booleans, all true)

$\text{result} \leftarrow$

▷ (a data structure)

for $i = 2, \dots, \lfloor \sqrt{n} \rfloor + 1$ **do**

if $p[i] = \text{true}$ **then**

$j \leftarrow i * i$

while $j \leq n$ **do**

$p[j] \leftarrow \text{false}$

$j \leftarrow j + i$

▷ Next multiple of i

for $i = 2, \dots, n$ **do**

if $p[i] = \text{true}$ add i to result

- Fijaos en que el algoritmo tiene un optimización: se empieza explorando en $j \leftarrow i * i$.

- Recuerda el laboratorio 2: un multiconjunto es un conjunto que permite elementos repetidos
- Internamente (en `MultiSetList`) se representó un multiconjunto como una lista de pares: `PositionList<Pair<E,Integer>>`
- La tarea consiste en implementar el método `equals`, dentro la clase `Utils`, que compara dos listas de pares:

```
boolean equals(PositionList<Pair<E,Integer>> l1,  
               PositionList<Pair<E,Integer>> l2)
```

utilizando recursión en lugar de iteración / bucles. El método `equals` debe devolver `true` si las listas contienen los mismos elementos (pero posiblemente en orden distinto).

- Recuerda que en l1 (y l2) no podría haber dos pares con elementos izquierdos idénticos, y que el elemento derecho siempre es > 0 . Se puede asumir que los elementos izquierdos nunca son `null`.

Reto No Obligatorio

Implementar una “Stack Machine”

Objetivos de este ejercicio:

- **Practicar con recursión y mapas (y algo de listas y pilas)**
- Dar una pequeña visión sobre cómo se puede implementar una máquina trabajando sobre una pila.
 - ▶ Lo haremos de forma similar a como se ejecuta el bytecode de Java.
- Notad que el tester normal – TesterLab4 – **no comprueba la funcionalidad del “Stack Machine”**
- Los tests del “Stack Machine” están definidos en la clase **Tests**, y son fáciles de entender.

Tarea: implementar una *stack machine* sencilla.

Una stack machine tiene una pila de enteros:

3
10
20
...

- En Java y aedlib:

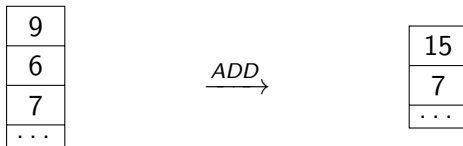
```
LIFO<Integer> stack;
```

- El código para ejecutar esta organizado en “procedures” (rutinas) cuyo código se puede obtener a través del Map:

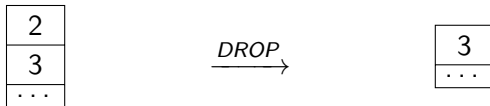
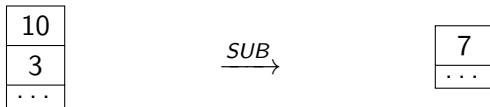
```
Map<String, PositionList<Instruction>> code;
```

Instrucciones del “stack machine” – Instruction

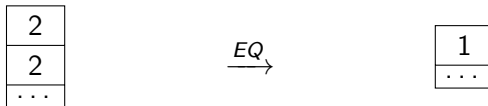
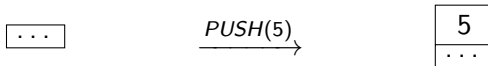
- La tarea es implementar un subconjunto de instrucciones de una *stack machine* que pueden calcular expresiones aritméticas básicas.
- La *stack machine* trabaja sobre una pila (stack) que contiene valores enteros.
- Las instrucciones cogen valores de la pila y ponen el resultado en la cima de la pila.
- Por ejemplo, la instrucción *add* suma los dos valores de la pila (9 y 6), y pone el resultado en la cima de la pila (15):



Instrucciones de la Máquina Virtual



Otras instrucciones de la Máquina Virtual



- *PRINT* imprime en la consola el entero que hay en la cima de la pila.
- *EQ* compara los dos enteros encima de la pila, y si son iguales deja el entero 1 encima de la pila, y si no deja 0 encima de la pila.

Instrucciones manipulando el flujo de control

- El “stack machine” ejecuta secuencialmente una lista de instrucciones:

`PositionList<Instruction> instrucciones`

- Existe instrucciones que cambian el orden de ejecución secuencial:
 - ▶ *CALL(procedure)*: ejecuta el código de la rutina *procedure*.
 - ▶ *RET*: termina la ejecución de una rutina.
 - ▶ *IF_SKIP(n)*, donde *n* es un entero positivo: si el entero encima de la pila es distinto de 0 salta *n* instrucciones y continua la ejecución.

Un programa para calcular y imprimir el factorial de 5

```
main:
```

```
[ PUSH(5), CALL("factorial"), PRINT, RET ]
```

```
factorial:
```

```
[ DUP, PUSH(1), EQ,  
  IF_SKIP(8),
```

```
// ELSE PART
```

```
DUP, PUSH(1), SWAP, SUB, CALL("factorial"),  
MULT, RET,
```

```
// THEN PART
```

```
DROP, PUSH(1), RET
```

```
]
```

- ¡Haz un simulacro en papel (con la pila) para entender cómo se ejecuta el programa!

Una pila en aedlib

- La interfaz es `upm.aedlib.lifo.LIFO<E>` describe una pila:

```
public interface LIFO<E> extends Iterable<E> {  
    ...  
    // Elimina y devuelve el elemento de la cima de la pila  
    public E pop();  
  
    // Devuelve el elemento encima de la cima de la pila  
    public E top();  
  
    // Añade e encima de la pila  
    public void push(E e);  
}
```

- Las clases `LIFOList` y `LIFOArray` implementan la interfaz `LIFO`.

Las Instrucciones

- Una instrucción, definida en la clase `Instruction`, tiene los métodos:
 - ▶ `getInstType()` – devuelve el tipo de instrucción (un enum sobre los nombres de instrucciones: `CALL`, `DROP`, etc)
 - ▶ `getNameParm()` – devuelve el parámetro `String` de la instrucción `CALL`
 - ▶ `getIntParm()` – devuelve el parámetro `int` de las instrucciones `PUSH` y `IF_SKIP`
- Ejemplo código para ejecutar *ADD*:

```
Instruction i = ...;  
if (i.getInstType() == Instruction.InstType.ADD) {  
    Integer e1 = stack.pop();  
    Integer e2 = stack.pop();  
    stack.push(e1+e2);  
}
```

La clase StackMachine

- Tiene un constructor

```
StackMachine(Map<String,PositionList<Instruction>> code)
```

que recibe el código de las rutinas como parámetro y crea la pila.

- Debe implementar el método

```
void run(String procedureName)
```

que ejecuta el código de la rutina procedureName. Si no hay ninguna rutina con este nombre lanza la excepción RuntimeException.

- Os recomendamos implementar algunos métodos auxiliares
- Una manera buena de depurar el “stack machine” es imprimir la siguiente instrucción i a ejecutar, y la pila:

```
System.out.println("ejecutando "+i+" stack: "+stack);
```

Notas Generales

- El proyecto debe compilar sin errores y debe cumplirse la especificación de los métodos a completar
- Debe pasar todos los test `TesterLab4` correctamente sin mensajes de error
- **Nota:** una ejecución sin mensajes de error y que pase todas las pruebas **no** significa que la implementación sea correcta (es decir, que funcione bien para cada posible entrada)
- Todos los ejercicios se corrigen manualmente antes de dar la nota final

¡Seguir estos consejos os permitirá conseguir mejores resultados!

- Corrección
- Ausencia de código repetido con la misma funcionalidad (podéis usar métodos auxiliares para evitarlo)
- Concisión y claridad del código
- Legibilidad, incluida selección de nombres descriptivos para variables y métodos
- El código debe estar correctamente indentado y con comentarios *útiles* cuando lo veáis necesario
- Eficiencia:
 - ▶ Se valorará la complejidad computacional del código
 - ▶ Se valorará no iterar innecesariamente en los recorridos de las estructuras de datos